

AUTOHAUS

Car Dealership Inventory & Sales Management System

Project Documentation

TDD Kata Submission — Full-Stack MERN Application

Node.js · Express · MongoDB · React · Tailwind CSS

Prepared as part of a Test-Driven Development technical assessment

1. Project Overview

AutoHaus is a full-stack car dealership inventory and sales management system built to satisfy a Test-Driven Development (TDD) kata. The application allows administrators to manage a vehicle inventory — adding, editing, restocking, and retiring vehicles — while customers can browse the live showroom, search and filter available stock, purchase vehicles, and download invoices for completed purchases.

The system was built as a MERN-stack application (MongoDB, Express, React, Node.js) with a strict separation between the presentation layer, the API layer, and the data layer, following clean architecture and SOLID principles throughout the backend.

2. Objectives

The kata required the project to demonstrate the following capabilities:

- A secure, token-based (JWT) authentication system supporting registration and login.
- A protected RESTful API for managing vehicle inventory, including search and filtering by make, model, category, and price range.
- Inventory-safe purchase and restock operations that never allow stock to fall below zero.
- Role-based access control distinguishing regular customers from administrators.
- A responsive single-page React application covering both customer and admin workflows.
- A backend built following the Red-Green-Refactor TDD cycle, with meaningful unit and integration test coverage.
- Transparent, documented use of AI tools throughout the development process, including AI co-authorship on commits.

3. Technology Stack

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS, React Router, Axios, Recharts
Backend	Node.js, Express 4, express-validator, JWT (jsonwebtoken), bcryptjs, PDFKit
Database	MongoDB Atlas via Mongoose ODM
Testing	Jest, Supertest, mongodb-memory-server
Tooling	ESLint, Nodemon, dotenv, Morgan (request logging)

4. System Architecture

The application follows a three-tier architecture. The React frontend communicates with the Express REST API exclusively over HTTPS/JSON, and the API in turn is the only layer permitted to read from or write to MongoDB. Within the backend, requests pass through a layered pipeline of routes, authentication and role-based middleware, controllers, and services, keeping business logic (such as stock validation and invoice generation) isolated and independently testable.

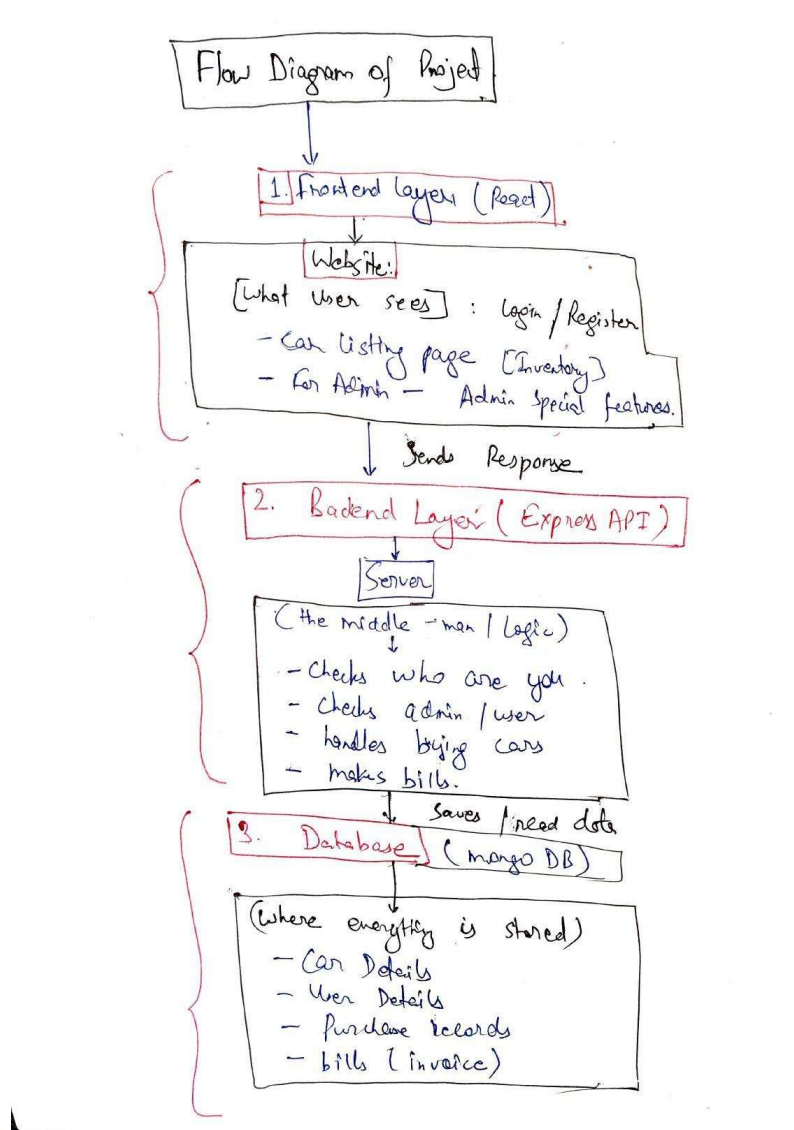


Figure 1 — Three-tier system architecture (frontend, backend, database)

4.1 Layer responsibilities

- Frontend (React SPA): renders the login/registration screens, the vehicle catalog, admin dashboards, and invoice views; stores the JWT and attaches it to outgoing requests via an Axios interceptor.

- Backend (Express API): validates the JWT and user role on every protected route, executes business rules (stock checks, purchase creation, invoice generation), and returns JSON responses.
- Data layer: MongoDB stores vehicles, users, purchases, and invoices; PDFKit generates the downloadable invoice documents from purchase records.

5. Application Flows

The diagrams below map the two primary user journeys through the system: authentication and role-based landing, and the end-to-end purchase workflow.

5.1 Authentication and role-based access

On visiting the site, a user is presented with a home page offering Sign In or Register. Existing users log in and are routed according to their role: administrators land on the admin console with access to inventory management, purchase records, invoices, and the analytics dashboard; regular customers land on the showroom to browse and purchase vehicles.

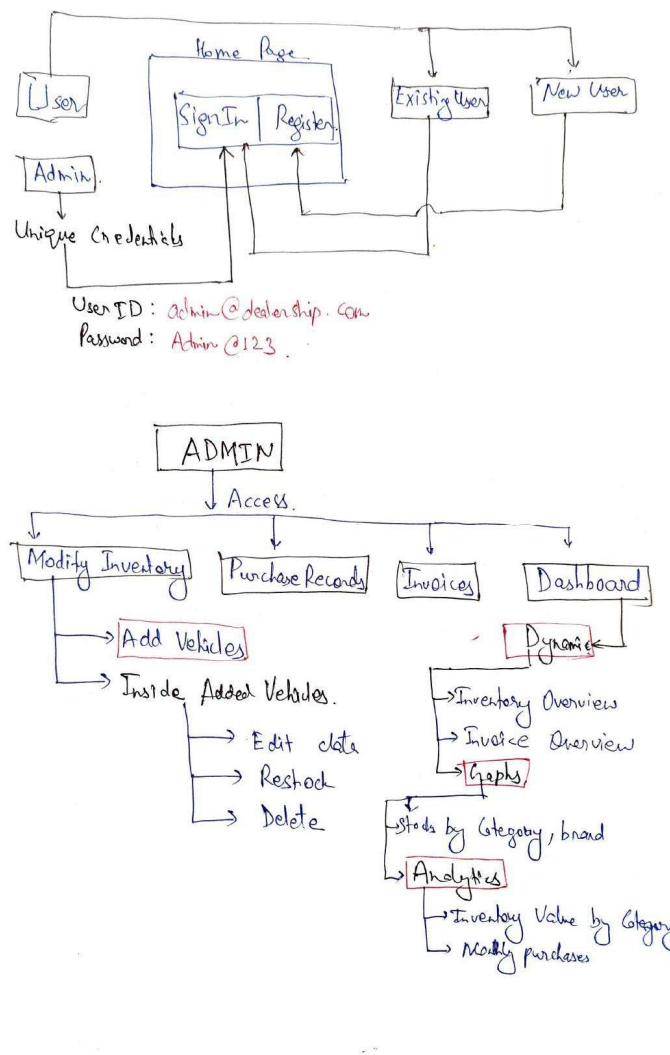


Figure 2 — Login/registration routing and admin access tree

5.2 Customer navigation and search

Once signed in, a customer can access the vehicle inventory, their purchase history, and their invoices. The inventory dashboard supports multi-parameter search — by vehicle name/model, fuel type, transmission, year, and category — as well as sorting (oldest, price ascending/descending, alphabetical) and minimum/maximum price filtering.

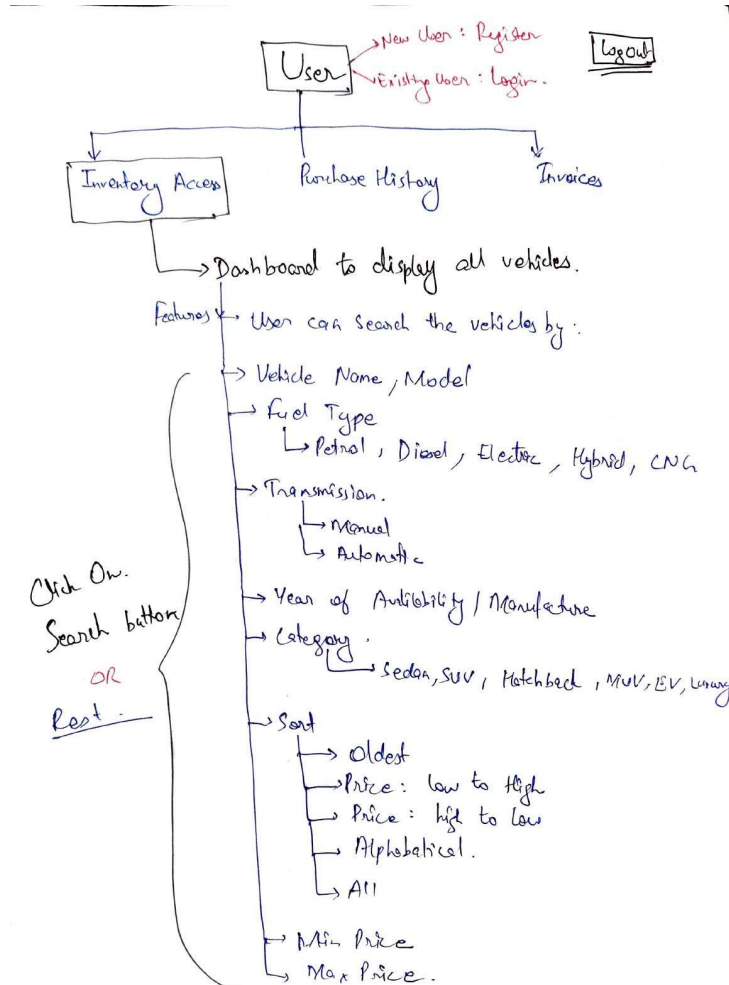


Figure 3 — Customer navigation and vehicle search/filter options

5.3 Purchase workflow

The purchase flow is the core transactional path of the system. A customer logs in, browses available vehicles, and selects one to purchase. The backend checks current stock before committing the transaction: if the vehicle is out of stock, the request is rejected; otherwise the purchase is recorded, the vehicle's stock is decremented, an invoice is generated, and the customer can download the resulting PDF.

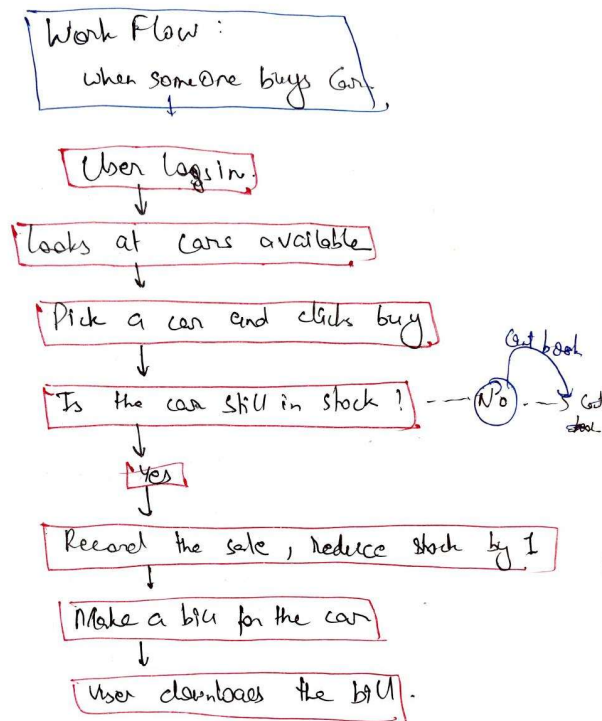


Figure 4 — End-to-end purchase workflow, including the stock-availability check

6. Feature Summary

6.1 Customer features

- Register and log in securely (JWT-based session).
- Browse the live vehicle showroom with pagination.
- Search by vehicle name/model, fuel type (Petrol, Diesel, Electric, Hybrid, CNG), transmission (Manual, Automatic), year, and category (Sedan, SUV, Hatchback, MUV, EV, Luxury).
- Sort results (oldest, price low-to-high, price high-to-low, alphabetical) and filter by minimum/maximum price.
- Purchase a vehicle — the purchase button is disabled automatically once stock reaches zero.
- View purchase history and download invoices as PDF.

6.2 Administrator features

- All customer-facing capabilities, plus:
 - Add, edit, restock, and delete vehicles.
 - View all purchase records across every customer.
 - View and manage invoices, including soft-deleting an invoice.
 - A dynamic analytics dashboard: inventory overview, invoice overview, stock-by-category/brand graphs, inventory value by category, and monthly purchase trends.

7. API Endpoints

7.1 Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account
POST	/api/auth/login	Authenticate and receive a JWT

7.2 Vehicles (protected)

Method	Endpoint	Description
POST	/api/vehicles	Add a new vehicle (admin only)
GET	/api/vehicles	List all available vehicles (paginated)
GET	/api/vehicles/search	Search by make, model, category, or price range
GET	/api/vehicles/:id	Retrieve a single vehicle's details
PUT	/api/vehicles/:id	Update a vehicle's details (admin only)
DELETE	/api/vehicles/:id	Delete a vehicle (admin only)

7.3 Inventory operations (protected)

Method	Endpoint	Description
POST	/api/vehicles/:id/purchase	Purchase a vehicle; decreases quantity by one
POST	/api/vehicles/:id/restock	Restock a vehicle (admin only)

7.4 Purchases, invoices, and analytics (protected)

Method	Endpoint	Description
GET	/api/purchases	List the current user's purchase history
GET	/api/invoices	List invoices for the current user
GET	/api/invoices/:id	Retrieve a single invoice
GET	/api/invoices/:id/download	Download an invoice as PDF
GET / DELETE	/api/admin/invoices	List / soft-delete invoices (admin only)
GET	/api/analytics/inventory	Inventory and sales analytics (admin only)

8. Testing Strategy

The backend was built following a strict Red-Green-Refactor cycle: failing test specifications were written first for each unit of behaviour, the minimal implementation was added to make them pass, and the code was then refactored without breaking the suite. Unit tests isolate controller logic; integration tests use Supertest against the Express app together with an in-memory MongoDB instance (mongodb-memory-server) to exercise real request/response cycles without touching a live database.

Test suite	File	Status	Coverage focus
Auth integration	auth.integration.test.js	Pass	Registration, password hashing, JWT issuance, role protection
Vehicle integration	vehicles.integration.test.js	Pass	RBAC, pagination, filtering, stock depletion, invoice PDF generation
Auth unit	auth.controller.test.js	Pass	Controller request/response contracts, duplicate email handling
Vehicle unit	vehicle.controller.test.js	Pass	CRUD operations, stock depletion safety, restock increments

Summary: 4 test suites passed, 35 tests passed (100% of the core business logic verified).

9. AI Usage Disclosure

In line with the kata's AI usage policy, AI tools were used throughout the development lifecycle as pair-programming aids. All AI-assisted commits carry a Co-authored-by trailer, and the full prompt history is recorded in PROMPTS.md at the root of the repository.

9.1 Tools used

- OpenAI Codex — implementation support, test execution, debugging, and documentation.
- Gemini / Google Antigravity — architecture planning, development environment support, and workflow iteration.
- Claude — backend and frontend scaffolding, test boilerplate, UI component drafts, and documentation support.

9.2 How AI was used

- Drafting the initial failing test specifications for authentication and vehicle endpoints before implementation (Red phase).
- Scaffolding Mongoose schemas, JWT middleware, and the purchase service, with stock-safety logic reviewed and adjusted by hand.
- Generating React component drafts and Tailwind styling, refined for the final UI/UX.
- Assisting with documentation, including this project write-up and the accompanying architecture and flow diagrams.

9.3 Reflection

AI tooling accelerated boilerplate-heavy work — test scaffolding, repetitive CRUD patterns, and initial UI markup — which freed up time to focus on the parts that actually required judgement: the stock-safety guarantees in the purchase service, role-based access control, and the overall API design. Every AI suggestion was reviewed, tested, and adjusted rather than accepted verbatim, keeping the TDD discipline intact.

10. Deliverables Checklist

- Public Git repository with a structured, descriptive commit history.
- README.md covering project explanation, local setup instructions (backend and frontend), screenshots, and the AI Usage section.
- PROMPTS.md containing the complete AI tooling prompt history.
- Automated test report (4 suites / 35 tests passing).
- This architecture and flow documentation, based on the author's own hand-drawn planning diagrams.